# Best Practices for Microservices Integration & Contract Evolution

*Decomposition of Consumption Ergonomics, Schema Hiding, Postel's Law, and Versioning Strategies*

**EXECUTIVE SUMMARY**

Seamless integration across microservice boundaries requires treating public APIs and event payloads as immutable digital contracts. Leaking internal database tables, exposing auto-increment keys, and making breaking contract modifications paralyze developer velocity. Applying Postel's Robustness Law and automated SDK generation guarantees frictionless cross-team collaboration.

## 1. THE THREE PILLARS OF SIMPLE CONSUMPTION

### 1. Type-Safe Client SDKs

Generate idiomatic client packages directly from OpenAPI specs, eliminating bespoke HTTP plumbing code.

### 2. Interactive API Portals

Host living Swagger UI / Redoc documentation with executable query examples and clear schema models.

### 3. Mocking Harnesses

Publish standalone Dockerized Prism / WireMock stubs enabling downstream teams to build and test offline.

## 2. NEVER EXPOSE STORAGE MODELS IN PUBLIC APIS



## 3. POSTEL'S ROBUSTNESS LAW IN MICROSERVICES

"Be conservative in what you send, be liberal in what you accept."

**Conservative Producer Rule**

Producers must strictly validate outgoing data against the formal JSON/Protobuf schema. Never emit malformed types or unexpected null values.

**Liberal Consumer Rule**

Consumers must configure deserializers to silently ignore unknown fields (`ignoreUnknownProperties = true`), allowing producers to add fields seamlessly.

## 4. ADDITIVE SCHEMA EVOLUTION RULES

| Contract Change | Modification Type | Compatibility Status |
|---|---|---|
| **Adding an Optional Field** | New field with default value (e.g. `postal_code: null`). | Fully Compatible No consumer impact. |
| **Renaming a Field** | Renaming `zip` to `postal_code`. | Breaking Must support both fields concurrently. |
| **Removing a Field** | Dropping an active field. | Breaking Requires 6-month deprecation cycle. |

## 5. API VERSIONING TOPOLOGIES

```
// URI Path Versioning (Recommended)
POST /api/v1/payments/charges
POST /api/v2/payments/charges  <-- Deployed in parallel; v1 deprecated gracefully

// HTTP Accept Header Versioning
GET /api/orders/99
Accept: application/vnd.enterprise.orders.v2+json
```

## 6. KAFKA SCHEMA REGISTRIES & EVENT COMPATIBILITY

Event-driven architectures require a centralized Schema Registry enforcing compatibility rules:

**Schema Registry Compatibility Modes**

- **BACKWARD:** New schema can read data written by previous schemas (new consumers can read old events).
- **FORWARD:** Old consumers can read data written by the new schema.
- **FULL:** Both backward and forward compatible (optimal for zero-downtime rolling deploys).

## 7. DECLARATIVE PACT CONSUMER CONTRACT TEST (GO)

```
// Pact Unit Test Defining Expected Integration Contract
func TestOrderServiceConsumer(t *testing.T) {
    pact := dsl.NewPact(dsl.Config{Consumer: "OrderService", Provider: "PaymentService"})
    defer pact.Teardown()

    pact.AddInteraction().
        Given("Customer has valid $100 balance").
        UponReceiving("A request to charge payment").
        WithRequest(dsl.Request{
            Method: "POST", Path: dsl.String("/api/v1/charge"),
            Headers: dsl.MapMatcher{"Content-Type": dsl.String("application/json")},
            Body: map[string]interface{}{"amount_cents": 5000, "customer_id": "CUST-100"},
        }).
        WillRespondWith(dsl.Response{
            Status: 200,
            Body: dsl.Match(map[string]interface{}{"status": "APPROVED", "tx_id": dsl.Like("tx_8812")}),
        })

    pact.Verify(func() error { return CallPaymentService(pact.Server.Port) })
}
```

## 8. PRODUCTION INTEGRATION VERIFICATION CHECKLIST

**Integration Standards Gates**

1. **No Auto-Increment IDs:** Enforce UUIDs or prefixed string IDs on all public entities.
2. **Tolerant Readers:** Enforce linter checks that all JSON parsers ignore unrecognized properties.
3. **Pact Verification in CI:** Every provider PR must verify all registered consumer contracts.
4. **Documentation Automation:** Automatically publish updated OpenAPI specs on every merge to master.